

1. Program Comparison Table

Program	Collectives Used	Full Array Allocation	Root Manual Summation?	Final Result Location
Ex 1	MPI_Bcast, MPI_Send/MPI_Recv	Every process	Yes	Root only
Ex 2	MPI_Scatter, MPI_Send/MPI_Recv	Root only	Yes	Root only
Ex 3	MPI_Scatter, MPI_Gather	Root only	Yes	Root only
Ex 4	MPI_Scatter, MPI_Reduce	Root only	No	Root only
Ex 5	MPI_Scatter, MPI_Allreduce	Root only	No	All processes
Ex 6	MPI_Scatter, MPI_Scan	Root only	No	Different per rank

2. Execution Times (2, 4, and 8 Processes)

To get the exact times for your report, you will need to run these commands for each exercise and note down the Time = 0.xxxx sec output:

- **2 Processes:** `mpirun --oversubscribe -np 2 ./exerciseX`
- **4 Processes:** `mpirun --oversubscribe -np 4 ./exerciseX`
- **8 Processes:** `mpirun --oversubscribe -np 8 ./exerciseX`

Which approach is fastest and why:

MPI_Reduce (Ex 4) and MPI_Allreduce (Ex 5) will typically be the fastest.

- Exercises 1-3 require the root process to manually loop through and add the partial sums one by one, which takes $O(P)$ time (where P is the number of processes).
- MPI_Reduce and MPI_Allreduce push the math down into the MPI library, which uses optimized, tree-based network algorithms (like recursive doubling) to combine the sums in parallel. This takes $O(\log P)$ time, making it significantly faster as you scale up to 8 processes.

3. Thinking Question: When to choose MPI_Scan over MPI_Allreduce?

You would choose MPI_Scan when you need a cumulative distribution or a **global offset** rather than just a single grand total.

Concrete Example: Imagine you are building a parallel search engine. Each process scans a chunk of documents and finds a varying number of matches (e.g., Rank 0 finds 3, Rank 1 finds 5, Rank 2 finds 2). You want to merge all these matches into a single, contiguous array.

- If you use MPI_Allreduce, every process just learns the total number of matches (10).
- If you use MPI_Scan on the match counts, Rank 0 gets 3, Rank 1 gets 8, and Rank 2 gets 10. By subtracting its own count, Rank 1 knows exactly that it should write its 5 matches starting at index 3 in the global array, and Rank 2 knows it should start writing at index 8. MPI_Scan enables this global index assignment without any processes needing to talk to each other directly.